SIMPLE PROCESSOR

Extended ISA Implementation Report

E/21/148 Ganathipan S. E/21/188 Jarshigan K.

1. Introduction

This report covers a full description of designing and developing a single cycle 8 bit processor for the CO224 Computer Architecture. This processor is capable of handling a minimum Instruction Set Architecture (ISA) of add, sub, and, or, mov, loadi, j, and beq instructions. Additionally, the bonus section adds mul, sll, srl, sra, ror, and bne to the ISA. Implementation was done with the assistance of Verilog HDL, with components that included an Arithmetic Logic Unit (ALU), an 8x8 register file, a control unit, a program counter, and a PC incrementer. This report is written for publication on GitHub, including careful documentation of the design process, instruction encodings, datapath changes, time analysis, test conditions, and bits of Verilog code. The source code and testbenches are available at the GitHub Repository.

2. Instruction Set Architecture

The processor uses a 32 bit fixed length instruction format, structured as follows:

1. **OP CODE (bits 31 - 24)**: Specifies the instruction operation.
2. **RD/IMM (bits 23 - 16)**: Destination register or immediate value (e.g., jump/branch offset or immediate operand).
3. **RT (bits 15 - 8)**: Source register or immediate value (e.g., branch offset).
4. **RS/IMM (bits 7 - 0)**: Source register or immediate value.

The opcodes for all instructions are defined in the following table:

Instruction	OPCODE	or	00000101
loadi	00000000	j	00000110
mov	00000001	beq	00000111
add	00000010	mul	00001100
sub	00000011	shift	00001101
and	00000100	bne	00001110

Table 1: OPCODE assignments for the processor's instruction set

The shift opcode (00001001) is shared by sll, srl, sra, and ror, with the specific shift type selected by bits [5:4] of the immediate field (bits 31 - 24):

1. 2'b01: Logical shift left (**sll**)
2. 2'b10: Logical shift right (**srl**)
3. 2'b11: Arithmetic shift right (**sra**)
4. 2'b00: Rotate right (**ror**)

3. Processor Architecture

The processor is designed as a single cycle architecture, where each instruction completes within one clock cycle (10 time units). The major components are:

1. **ALU**: Performs arithmetic and logical operations, extended to support multiplication and shifts.
2. **Register File**: An 8x8 array storing eight 8 bit registers, supporting asynchronous reads and synchronous writes/resets.
3. **Control Unit**: Decodes instructions and generates control signals for the ALU, register file, and PC.
4. **Program Counter (PC)**: Maintains the address of the next instruction.
5. **PC Incrementer**: Computes the next PC value, including branch/jump targets.

The block diagram of the processor is shown in Figure 1.

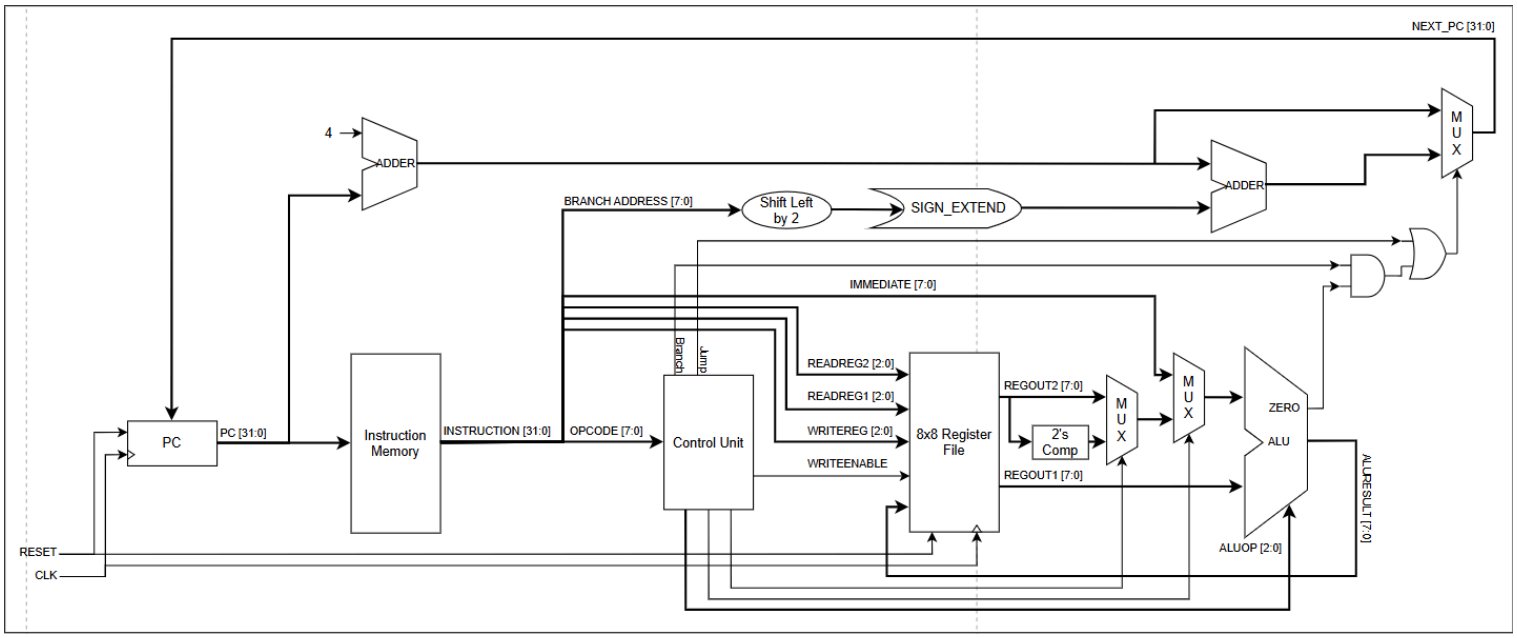

Figure 1: Block diagram of the 8 bit single cycle processor

4. Implementation Details

4.1 ALU Unit

The top level ALU module (ALU.v) is designed to receive two 8 bit operands (DATA1, DATA2), a 3 bit operation selector (SELECT), and output an 8 bit result (RESULT) along with a zero flag (ZERO). Depending on the selector input, the ALU activates one of the submodules:

- forwardUnit
- addUnit
- andUnit
- orUnit
- mulUnit
- shifterUnit

These submodules are instantiated within the aluUnit and are selected via a case structure.

4.1.1 forwardUnit

- **Purpose**: Forwards the input DATA2 directly to RESULT.
- DATA2→ RESULT

4.1.2 addUnit

- **Purpose:** Performs the addition of DATA1 and DATA2.
- DATA1 + DATA2 → RESULT

4.1.3 andUnit / orUnit

- **Purpose:** Bitwise AND / OR operations.
- DATA1 &/| DATA2 → RESULT

4.1.4 mulUnit

- **Purpose:** Performs multiplication using a simplified version of Booth's Algorithm.
- DATA1 × DATA2 → RESULT[7:0]
- **Details:**
  - Iterates over each bit of DATA2.
  - Shifts and adds DATA1 when the corresponding bit is 1.
  - Truncates to the lower 8 bits.

4.1.5 shifterUnit

- **Purpose:** Supports 4 types of bit shift operations using conca:
  - sll: DATA1 << n → RESULT
  - srl: DATA1 >> n (logic) → RESULT
  - sra: DATA1 >>> n (arithmetic) → RESULT
  - ror: Rotate right DATA1 by n → RESULT
- **Control Field:** DATA2[5:4] defines shift type; DATA2[3:0] defines shift amount.
- **Implementation Note:** Shift operations are implemented using Verilog concatenation.

The aluUnit uses a case statement to choose the result from one of the submodules based on the 3 bit SELECT input. The ZERO flag is calculated from the result of the adder.

SELECT	Function	Supported Instruction	Unit's Delay
000	fwdOut	Loadi, mov	#1
001	sum	add, sub	#2
010	andOut	and	#1
011	orOut	or	#1
100	mulOut	mul	#3
101	shiftOut	sll, srl, sra, ror	#4

Table 2: Case statements for ALU unit

4.1.6 ZERO Flag

- Used for branch instructions like beq and bne.
- Set to 1 if the result of the addition (or subtraction modelled using 2's complement) is zero.

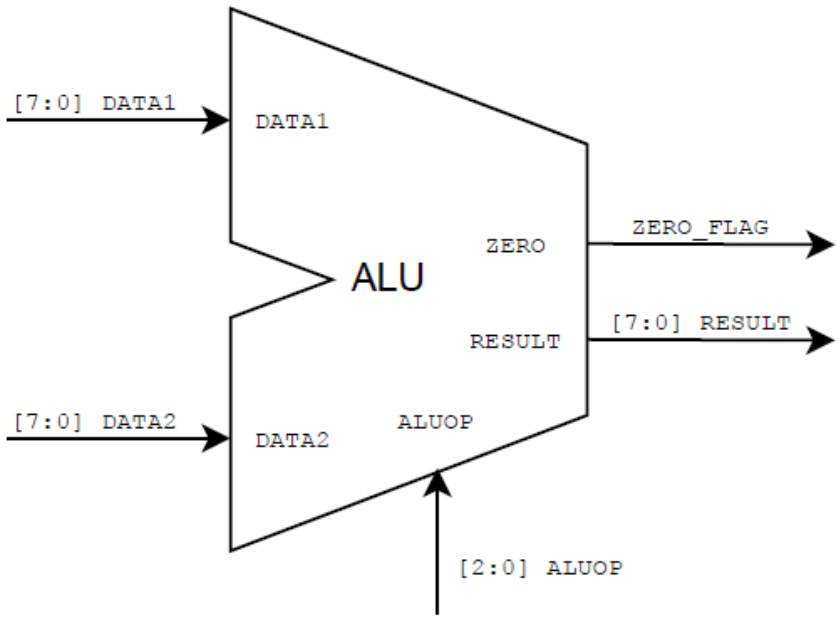

Figure 2: ALU

4.2 Register File

The register file (RegisterFile.v) stores 8 bit data values in eight registers and supports both read and write functionalities. It is an essential component for the temporary storage of operand values during instruction execution.

4.2.1 Reset Functionality

The register file contains 8 registers, each 8 bits wide. It supports a synchronous reset mechanism. When the RESET signal is asserted (high) on a rising clock edge, all registers are cleared and set to 0. This ensures that the processor begins execution from a clean state.

4.2.2 Timing Delays

- **Read Delay:** When reading from a register (via OUT1ADDRESS or OUT2ADDRESS), the output appears after a delay of **#2 time units**.
- **Write Delay:** When a register is written to on the rising edge of the clock (if WRITE is enabled and RESET is low), the data is updated after a **#1 time unit** delay.
- **Reset Delay:** When RESET is high, the entire register array is cleared with a **#1 time unit** delay per register, synchronized with the clock.

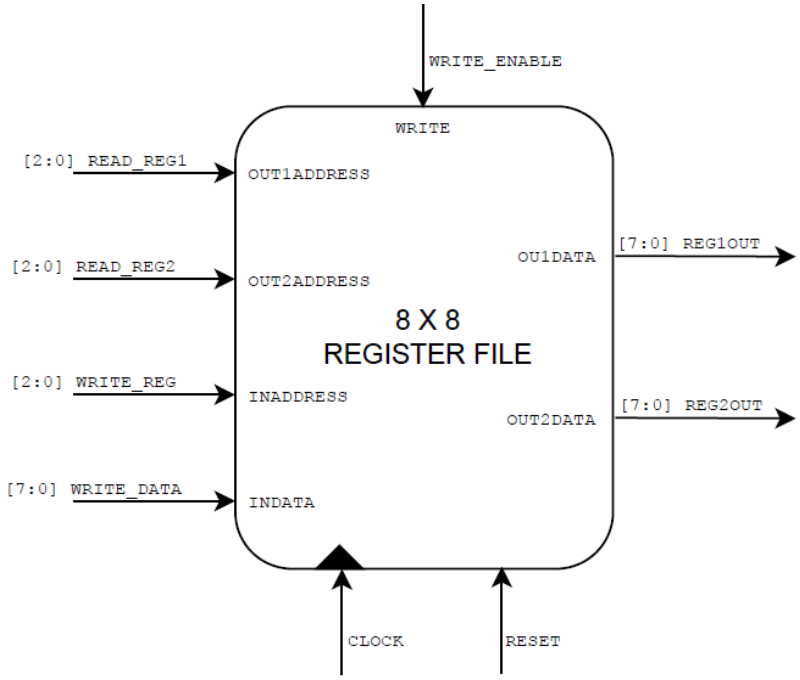

Figure 3: Register File

4.3 Control Unit

The control unit is responsible for decoding the instruction opcode and generating the necessary control signals that coordinate the operation of the ALU, register file, program counter, and branching mechanisms.

4.3.1 Functionality

The control unit receives an 8 bit opcode input and, based on its value, generates the following control signals:

- **ALUOP (3 bits):** Determines the operation the ALU should perform (e.g., add, and, or, shift, multiply).
- **writeEn (1 bit):** Enables writing to the register file.
- **signControl (1 bit):** Selects whether the operand should be two's complement (used in subtraction).
- **operandControl (1 bit):** Selects between immediate and register based operands.
- **branchControl (2 bits):** Encodes branching logic: no branch (00), branch if equal (01), branch if not equal (10).
- **jumpControl (1 bit):** Enables unconditional jump.

4.3.2 Timing Delays

Decode Delay: The control unit introduces a **#1** time unit delay after an opcode change before the outputs are updated.

4.4 Program Counter

4.4.1 Function

The Program Counter is responsible for holding the address of the current instruction. At every rising edge of the clock, it updates this address to either the next instruction or the target address computed by the PC Incrementer. It includes a synchronous reset feature that ensures the program restarts from address zero when needed.

4.4.2 Reset

- The PC is cleared to zero synchronously when the reset signal is active.
- This allows the processor to begin instruction fetching from the first address after a reset.

4.4.3 Timing Delays

- The PC introduces a **#1 time unit delay** on every update.

4.5 PC Incrementer

4.5.1 Function

The PC Incrementer calculates the next value of the program counter. It supports regular sequential instruction execution (PC + 4), unconditional jumps, and conditional branching based on the branch control signal and the ALU's ZERO output.

4.5.2 Branching and Jumping Logic

- For sequential execution, the incrementer simply adds 4 to the current PC.
- For jump instructions, the new PC is computed by sign extending the 8 bit branch address and shifting it by 2 to maintain word alignment.
- For conditional branches like beq and bne, the PC is updated only if the conditions (equality or inequality) are met, based on the ZERO signal.

4.5.3 Timing Delays

- The incrementer introduces a **#1 time unit delay** for computing PC + 4.
- An additional **#2 time unit delay** is introduced when calculating the full offset and resulting PC for jump or branch instructions.

The PC module receives its next address input from the PC Incrementer, which calculates the target based on current program flow requirements. The control unit directs whether the processor should proceed with sequential instruction fetching, perform a jump, or branch conditionally based on comparison results. Working in tandem, the PC and PC Incrementer ensure accurate and timely updates to the instruction address, thereby managing the program's control flow and maintaining correct sequencing for all types of instructions.

5. Integration & Control (CPU)

This module encapsulates the entire datapath of the 8 bit single cycle processor by integrating the program counter, control unit, ALU, register file, and program incrementer.

- **Instruction Fetch:** The ProgramCounter fetches the instruction from memory using PC_OUT, which is used in the testbench to provide the next instruction.
- **Control Signal Decoding:** The control unit interprets the opcode bits [7:0] from the instruction and generates necessary signals to control the datapath.

- **Operand Fetch:** The reg_file fetches two register operands using fields [18:16] and [26:24], while writing the ALU result to [10:8] when enabled.
- **ALU Operand Selection:**
    - If OPERAND_CONTROL is high, the second operand is taken directly as an 8 bit immediate from [31:24].
    - If SIGN_CONTROL is enabled, the second operand is sign extended using two's complement.
- **ALU Execution:** The ALU unit performs the required computation and outputs both the result and a ZERO flag.
- **Control Flow Update:** The pc Incrementer decides the next value of the PC based on whether the instruction is a normal, branch, or jump.

6. Data Path & Data Flow

The datapath of this 8 bit single cycle processor consists of modular arithmetic and control components that are interconnected to execute an instruction in one complete cycle. It covers instruction fetch, decode, operand access, arithmetic/logical execution, register write back, and program counter control.

6.1 Key Modules

- **Program Counter (PC):** Supplies the current instruction address.
- **Instruction Memory:** (Handled via testbench) Provides a 32 bit instruction at the given address.
- **Control Unit:** Decodes the instruction and issues control signals.
- **Register File:** Provides two read ports and one write port for operands.
- **ALU:** Perform operations like add, and, or, mul, shift, and forward.
- **PC Incrementer:** Computes PC + 4 and handles branching and jumping.

6.2 Data Flow

1. **Fetch**: PC_OUT is used to fetch the instruction.
2. **Decode**: Control signals (ALUOP, WRITE_ENABLE, etc.) are derived from OPCODE.
3. **Read**: Operands are read from the register file using OUT1ADDRESS and OUT2ADDRESS.
4. **ALU Routing**:
    - DATA1 is taken directly from the register.
    - DATA2 is either from the register or from the immediate field, depending on OPERAND_CONTROL.
    - For subtraction or comparison (BEQ/BNE), SIGN_CONTROL activates two's complement transformation.

5. **Execute**: The ALU performs operations. For MUL and SHIFT, the respective ENABLE signals activate their execution.
6. **Write Back**: The ALU result is written back to the register file if WRITE_ENABLE is asserted.
7. **PC Update**: The next PC value is calculated by pcIncrementer based on BRANCH_CONTROL, JUMP_CONTROL, and the ZERO flag.

The control unit dynamically enables the correct data paths and modules:

- Enables writing back results.
- Choose the operand source.
- Triggers branch/jump logic.
- Route the correct operation result to the ALU output.

7. Instruction Set Architecture (ISA)

7.1 J (Jump)

The j instruction performs an **unconditional jump** to a new instruction address calculated using the immediate value provided in the instruction. This allows the processor to change control flow and begin execution from a different part of the instruction memory.

- The immediate field (bits [23:16]) contains an 8 bit signed offset, which is **sign extended** to 32 bits.
- The offset is then **left shifted by 2 bits** to ensure word alignment (since each instruction is 4 bytes).
- The resulting value is **added to PC + 4** to compute the **target address**.
- This computation is handled by the **PC Incrementer**, which has an additional adder dedicated to branch/jump instructions.

When executing j, the control unit:

- Sets JUMPCONTROL = 1 to enable the jump logic.
- Ignores register reads and the ALU, as the jump doesn't depend on register contents or arithmetic.

7.1.1 Timing Diagram

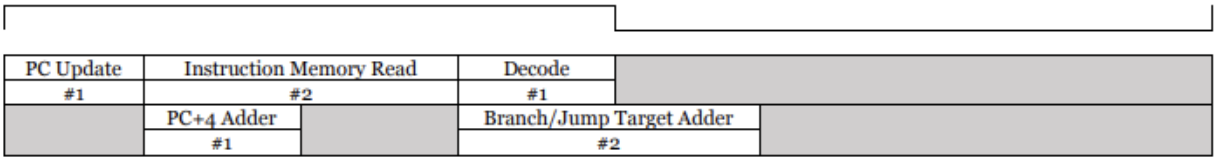

Figure 4: Timing diagram of j

7.2 BEQ (Branch If Equal)

The beq instruction enables **conditional branching based on equality**. It compares two values from registers and causes a branch (jump) if the values are **equal.**

- The instruction reads values from two **source registers**: RS and RT.
- These values are compared by **subtracting** one from the other using the ALU's **adder unit**.
- If the result of the subtraction is **zero**, the ALU's ZERO flag will be **1**.
- The control unit interprets ZERO = 1 as a signal to **take the branch**.

The branch target is computed as follows:

- The branch offset is taken from the instruction's **bits [23:16]**, which form an **8 bit signed value**.
- This offset is **sign extended** to 32 bits to allow forward and backwards branching.
- The result is **left shifted by 2 bits** (i.e., multiplied by 4) for **word alignment**, since instructions are 4 bytes each.
- The offset is then **added to PC + 4** to compute the final **branch target address**.

7.2.1 Timing Diagram

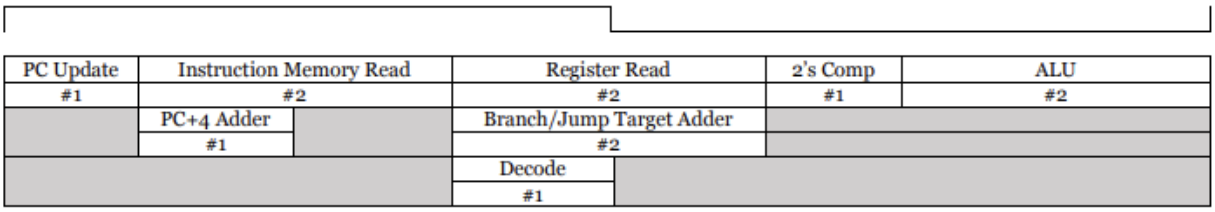

Figure 5: Timing diagram of beq

7.3 BNE (Branch If Not Equal)

The bne instruction enables **conditional branching** based on inequality. It compares two values from registers and causes a branch (jump) if the values are **not equal**. This is useful for implementing control flow decisions such as conditional loops or skipping code blocks.

- The instruction reads values from two source registers (RS and RT).
- These values are compared by subtracting one from the other using the **ALU's adder unit**.
- If the result of the subtraction is **non zero**, the **ZERO** flag output from the ALU will be **0**.
- The control unit then interprets ZERO = 0 as a signal to **branch**.

The branch target is computed as follows:

- The **immediate offset** (8 bit signed value from bits [23:16]) is **sign extended** to 32 bits.
- The value is **left shifted by 2 bits** (×4) for word alignment.
- The result is added to PC + 4 to compute the **target address**.
- The **PC Incrementer** uses this result as the next PC only if the bne condition is satisfied.

7.3.1 Timing Diagram

PC Update	Instruction Memory Read		Register Read		2's Comp	ALU
#1	#2		#2		#1	#2
	PC+4 Adder		Branch/Jump Target Adder			
	#1		#2			
			Decode			
			#1			

Figure 6: Timing diagram of bne

7.3.2 Data Path

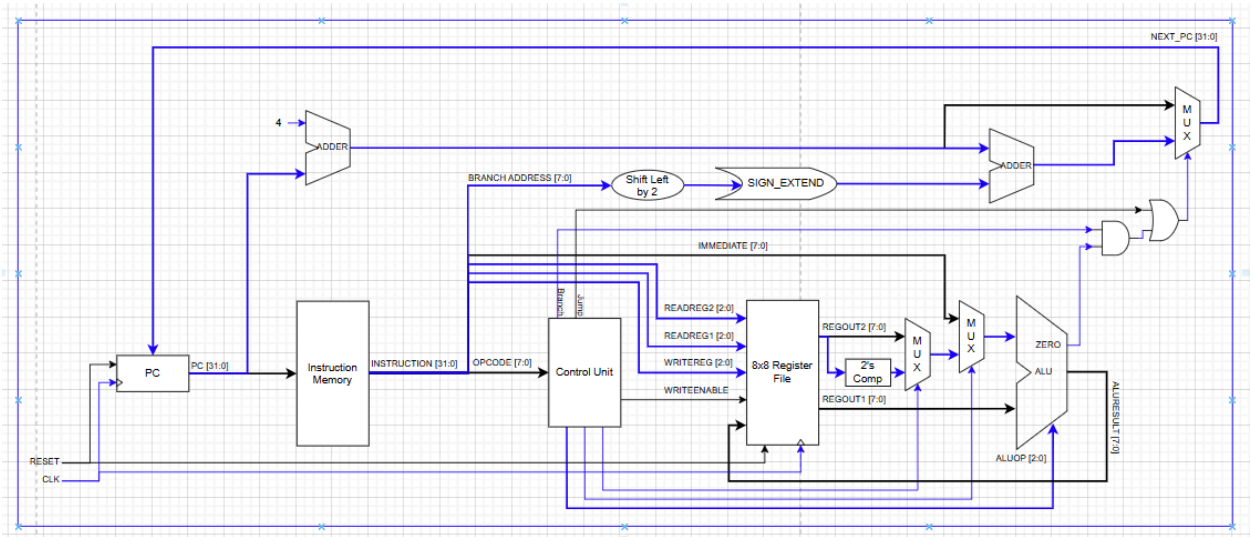

Figure 7: Datapath diagram for bne

7.4 MUL (Multiplication)

Multiplies the values in two source registers (Rs and Rt) as **signed 8 bit integers**, and stores the **lower 8 bits** of the signed product in the destination register (Rd). The multiplication is handled by a dedicated mulUnit, which performs **signed 8 bit × 8 bit multiplication** using iterative addition and shifting. The algorithm is designed as follows:

- DATA1 (multiplicand) is extended to 16 bits and stored in temp2.
- For each bit set in DATA2 (multiplier), the value of temp2 is added to a running total of temp1.
- temp2 is shifted left by 1 each iteration, simulating standard binary multiplication.
- Only the **least significant 8 bits** (temp1[7:0]) are assigned to RESULT.

7.4.1 Timing Diagram

PC Update	Instruction Memory Read		Register Read		ALU
#1	#2		#2		#3
	PC + 4 Adder		Decode		
	#1		#1		
Register Write					
#1					

Figure 8: Timing diagram of mul

7.4.2 Data Path

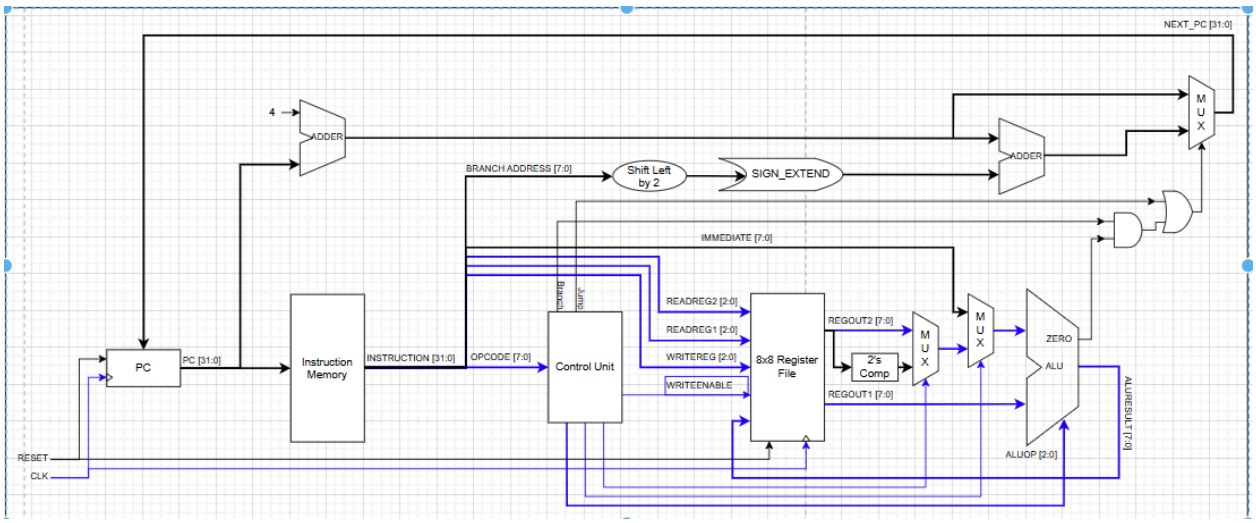

Figure 9: Datapath diagram for multiplication

7.4.3 Diagram

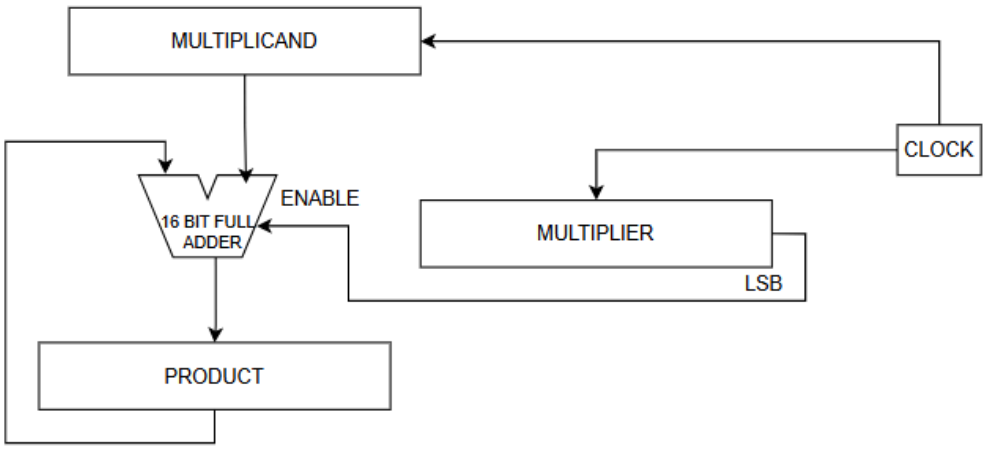

Figure 10: Diagram for multiplication

7.5 SLL (Shift Logical Left), SLR (Shift Logical Right), SRA (Arithmetic Shift Right), ROR (Rotate Right)

These are handled by the shared shifter unit.

Implementation

7.5.1 Instruction Fetch

- **#1:** Program Counter (PC) is updated.
- **#2:** Instruction is fetched from instruction memory.

7.5.2 Register File Read

- **#4:** Value in RT is read from the register file after a **#2** time unit delay.

7.5.3 Control Unit Decode

- Opcode is detected (00001101).
- **#1 delay** to generate control signals:
    - ALUOP = 101 activates the shifter unit.
    - OPERANDCONTROL = 1 selects the immediate as the second operand (contains shift type and amount).
    - WRITEENABLE = 1 result will be written to the register.

7.5.4 Immediate Field Extraction

- From instruction bits [7:0]:

    - DATA2[5:4]: shift type
    - DATA2[3:0]: shift/rotate amount

7.5.5 ALU Operation:

The shifterUnit executes one of the following, based on DATA2[5:4]:

Type	Code	Operation
SLL	00	Logical left shift, insert 0 right
SRL	01	Logical right shift, insert 0 left
SRA	10	Arithmetic right shift, replicate MSB
ROR	11	Rotate right, wrap LSB to MSB

Table 3: OP Codes for Shift Types

- Executes a loop: 1 shift per iteration.
- Artificial **#4** time unit delay for full operation.

7.5.6 How Each Operation Is Calculated

7.5.6.1 SLL

Every bit in the 8 bit input is shifted one place to the **left**. The **MSB (bit 7)** is discarded, and a **zero is inserted into the LSB (bit 0)**.

7.5.6.2 SLR

All bits are shifted right. The LSB (bit 0) is discarded, and a zero is inserted into the MSB (bit 7).

7.5.6.3 SLR

Similar to SRL, except that the **sign bit (MSB)** is preserved. Each shift replicates the MSB into the next position, maintaining the **signedness** of the value.

7.5.6.4 ROR

All bits are shifted one position **to the right**, but **the bit shifted out from the LSB wraps around to the MSB**. No bits are lost.

7.5.7 Write Result to RD

- ALU result written to register RD at the next clock edge.
- Delayed by **#1** time unit.

7.5.8 PC Update

- PC updated to **PC + 4** using an adder.

7.5.9 Timing Diagram SLL

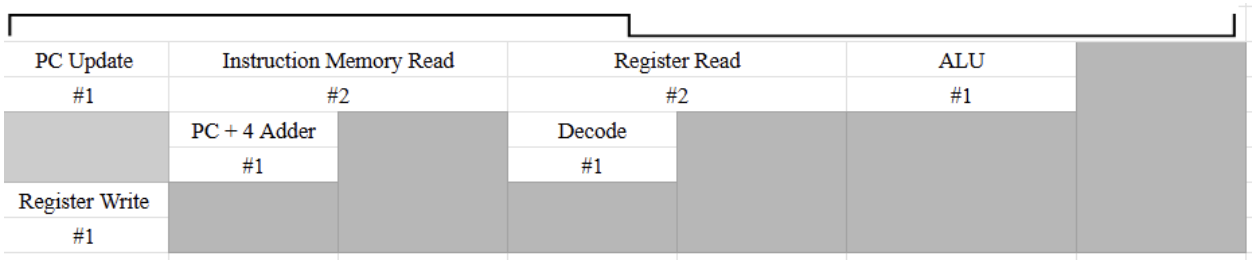

Figure 11: Timing diagram of sll

7.5.10 Timing Diagram SLR

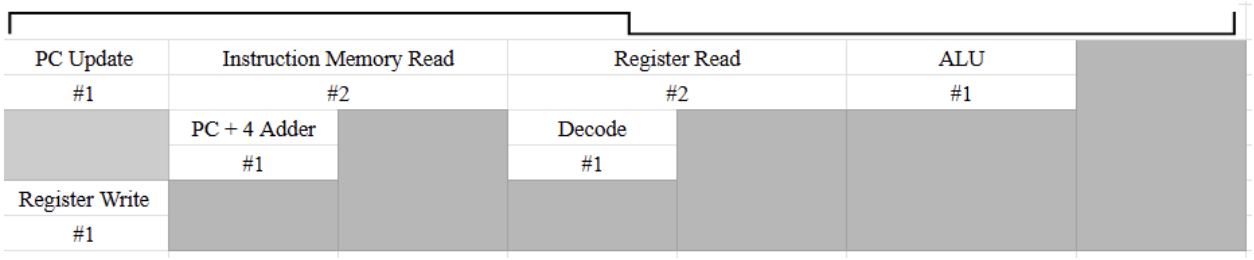

Figure 12: Timing diagram of slr

7.5.11 Timing Diagram SAR

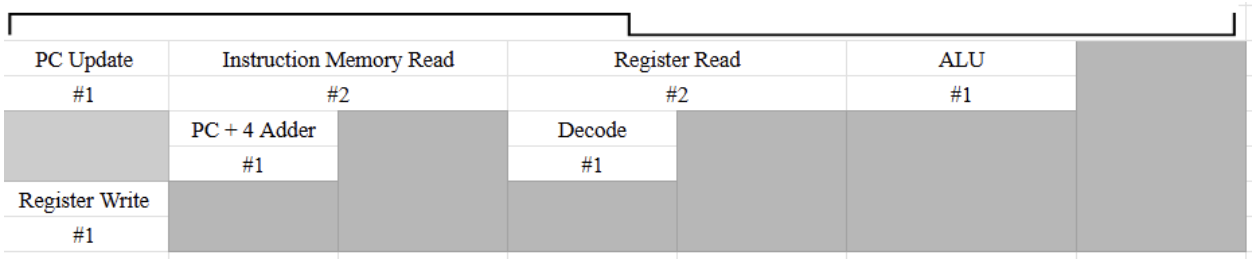

Figure 13: Timing diagram of sar

7.5.12 Timing Diagram ROR

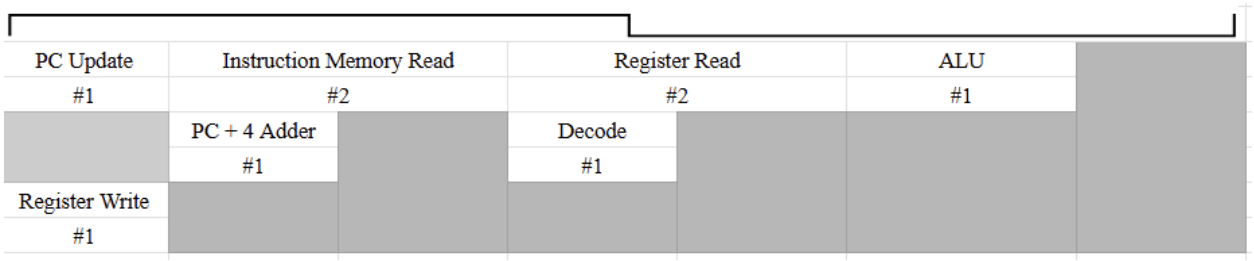

Figure 14: Timing diagram of ror

7.5.12 Diagram

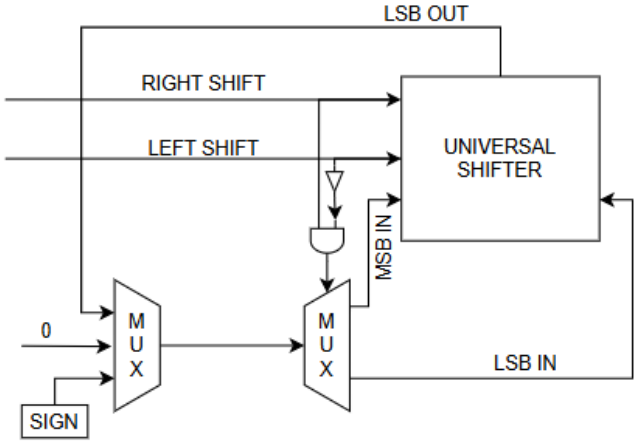

Figure 15: Diagram for shift

8. Gate Level Implementations

In this processor, we were instructed to implement arithmetic and logic operations using **only gate level Verilog** modules. Two such components were written at a low level: the shifterUnit and mulUnit. These units **avoid high level operators** such as *, <<, and>>, and instead use **bitwise operations, concatenation**, and **manual loops** to simulate hardware behaviour as in real integrated circuits.

8.1 Multiplier

This module performs **unsigned 8 bit × 8 bit multiplication** using a **sequential approach**. It uses **shift registers, manual bit checking, and a full 16 bit ripple carry adder built with only AND/OR/XOR gates**. The result is the **lower 8 bits** of the 16 bit product.

8.1.1 Reset

- All internal registers and outputs are cleared
- regA, regB, result, count, and done are all set to 0

8.1.2 Start Signal

- Loads inputs a and b:
  - regA = {8'b0, a} → multiplicand placed in lower 8 bits

    - regB = {8'b0, b} → multiplier in lower 8 bits; upper bits used as accumulator
- adder_out = 0, count = 0, done = 0

8.1.3 Main Loop

Each cycle does the following:

a. Check LSB of regB

- If the **lowest bit** of the multiplier (regB[0]) is 1, we **add regA to adder_out**
- This mimics the classic **shift and add multiplier** algorithm

b. Shift regA Left by 1:

- Simulates multiplying by 2
- No << used; instead, done using **concatenation** to shift bits left

c. Shift regB Right by 1:

- Simulates dividing by 2.
- LSB is consumed, and the next bit will be checked on the next cycle.

d. After 8 Cycles:

- The lower 8 bits of adder_out are saved in the result.
- The done flag goes high.

8.1.1 Full adder 16bit Function

The multiplication relies on a **custom full 16 bit adder** implemented inside the module. Here's how it works:

- Uses a for loop from 0 to 15.
- For each bit:
    - Computes sum = a ⊕ b ⊕ carry.
    - Computes carry_out = majority(a, b, carry).
        - Implemented as carry = (a & b) | (a & carry) | (b & carry).
- This ripple carry structure uses **only AND, OR,** and **XOR** logic.
- Equivalent to chaining 16 instances of a 1 bit full adder.
- No + operator is used.

8.2 Shifter

This module performs **8 bit shifting and rotating operations** using only **basic logic gates** and **manual bit control**. It does not use any built in shift operators or arithmetic expressions. Instead, it processes one shift **per clock cycle**, just like a real hardware **serial shift register**.

8.2.1 Startup and Setup

- When the reset signal is active, the internal registers are cleared.
- When the start signal is given, the circuit takes in the data, shift type, and shift amount.
- A counter is started to control how many shifts are done.

8.4.2 Control Decoding Using Logic Gates

- Instead of using if or switch statements, the module uses **NAND and AND gates** to detect which shift operation is selected.
- This is how control selection works in real chips using **logic gate decoders**.

8.4.3 Manual Bit Shifting

- During each clock cycle:
    - The module performs **one shift** by **manually moving each bit** to its new position.
    - For example, in a logical left shift, each bit is moved one place to the left, and the rightmost bit is filled with 0.
    - In a rotate right operation, the last bit is wrapped around to the first bit (circular shift).
    - For arithmetic right shift, the leftmost bit (sign bit) is copied to keep the sign of negative numbers.

8.4.4 Loop and Completion

- The circuit repeats this shifting process for the number of times specified by the shift amount.
- After the final shift:
    - The result is stored in the output register
    - A **done** signal is raised to indicate that shifting is complete

9 GTK Wave

9.1 MUL

```
loadi  r1, 0x03        // r1 = 3
loadi  r2, 0x01        // r2 = 1
mul    r5, r3, r4      // r5 = r1 * r2
```

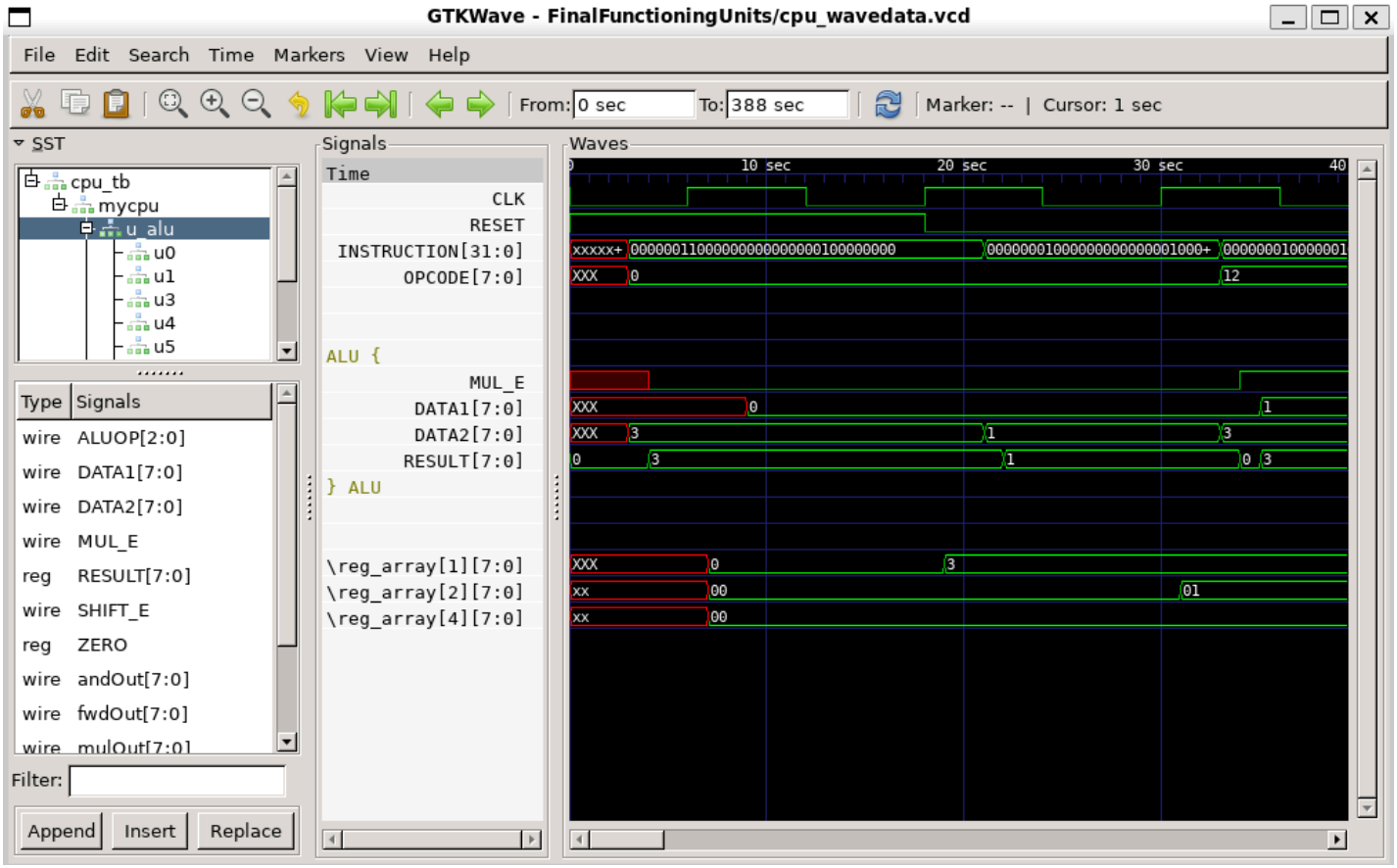

Figure 16: GTK Wave for mul

9.2 SLL

```
loadi  r1, 0x03        // r1 = 3
sll    r4, r1, #1      // r4 = r1 << 1
```

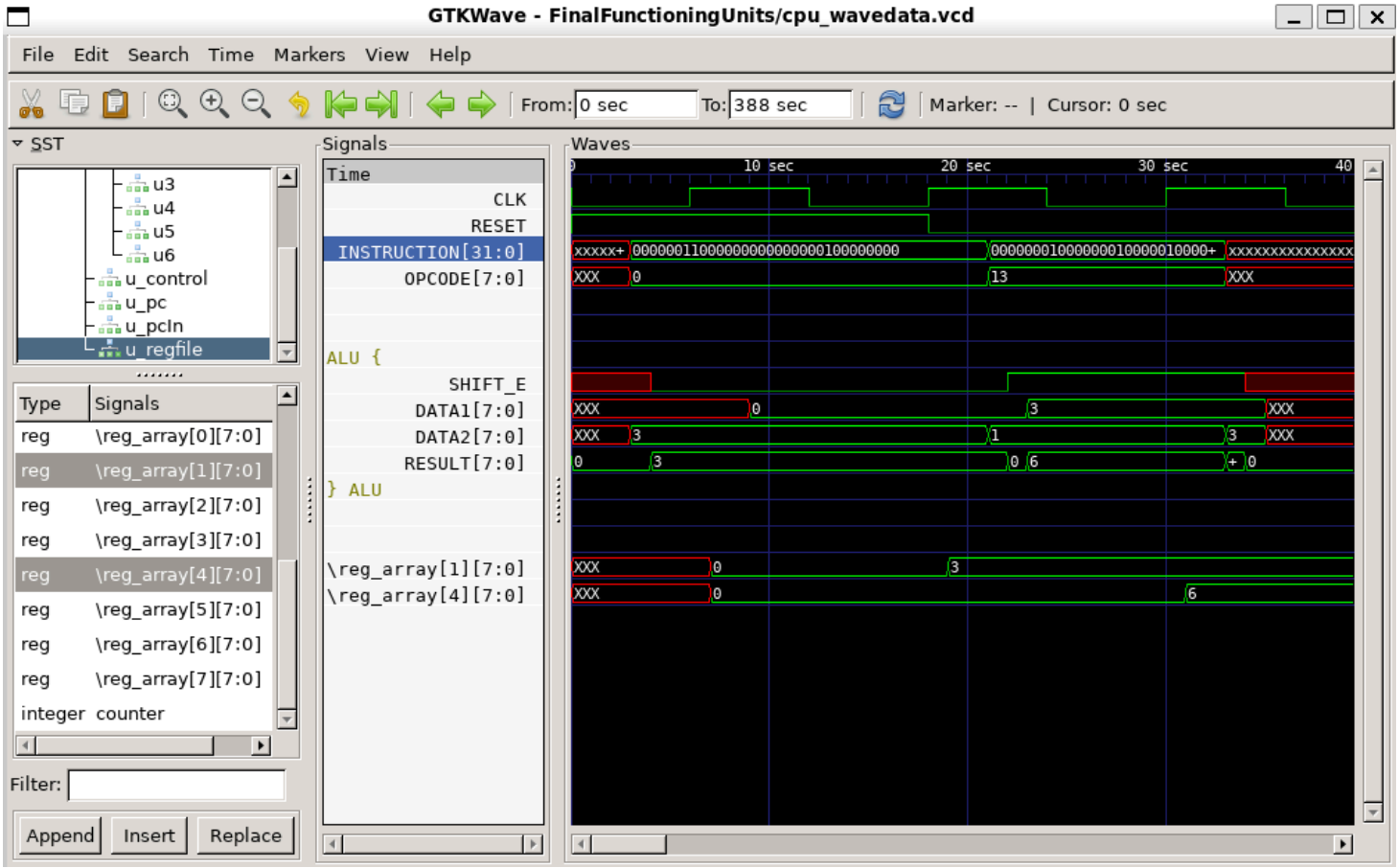

Figure 17: GTK Wave for sll

9.3 SRL

```
loadi   r1, 0x03        // r1 = 3
srl     r4, r1, #1      // r4 = r1 >> 1
```

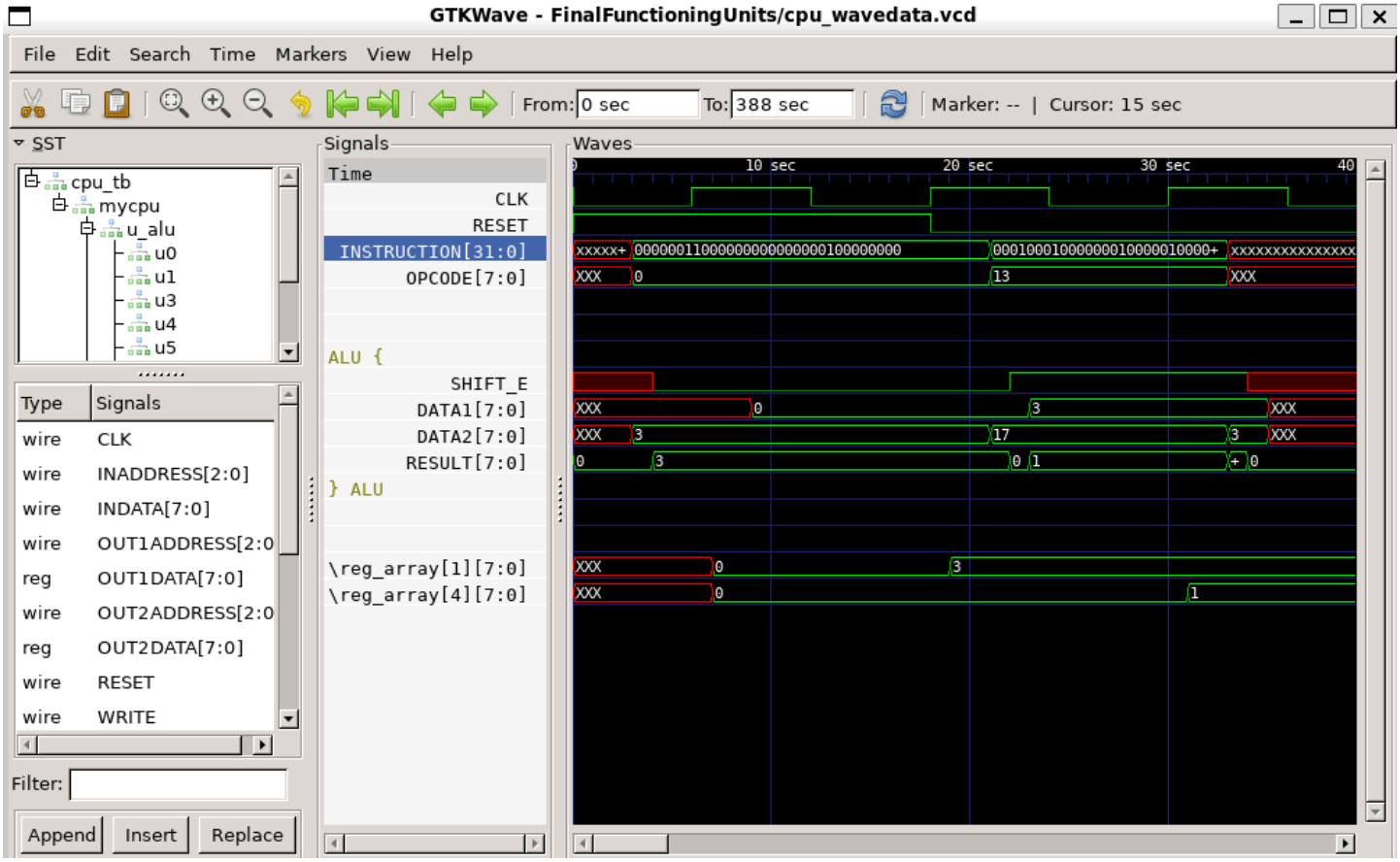

Figure 18: GTK Wave for srl

9.4 SRA

```
loadi  r1, 0x03        // r1 = 3
sra    r4, r1, #1      // r4 = r1 >>> 1
```

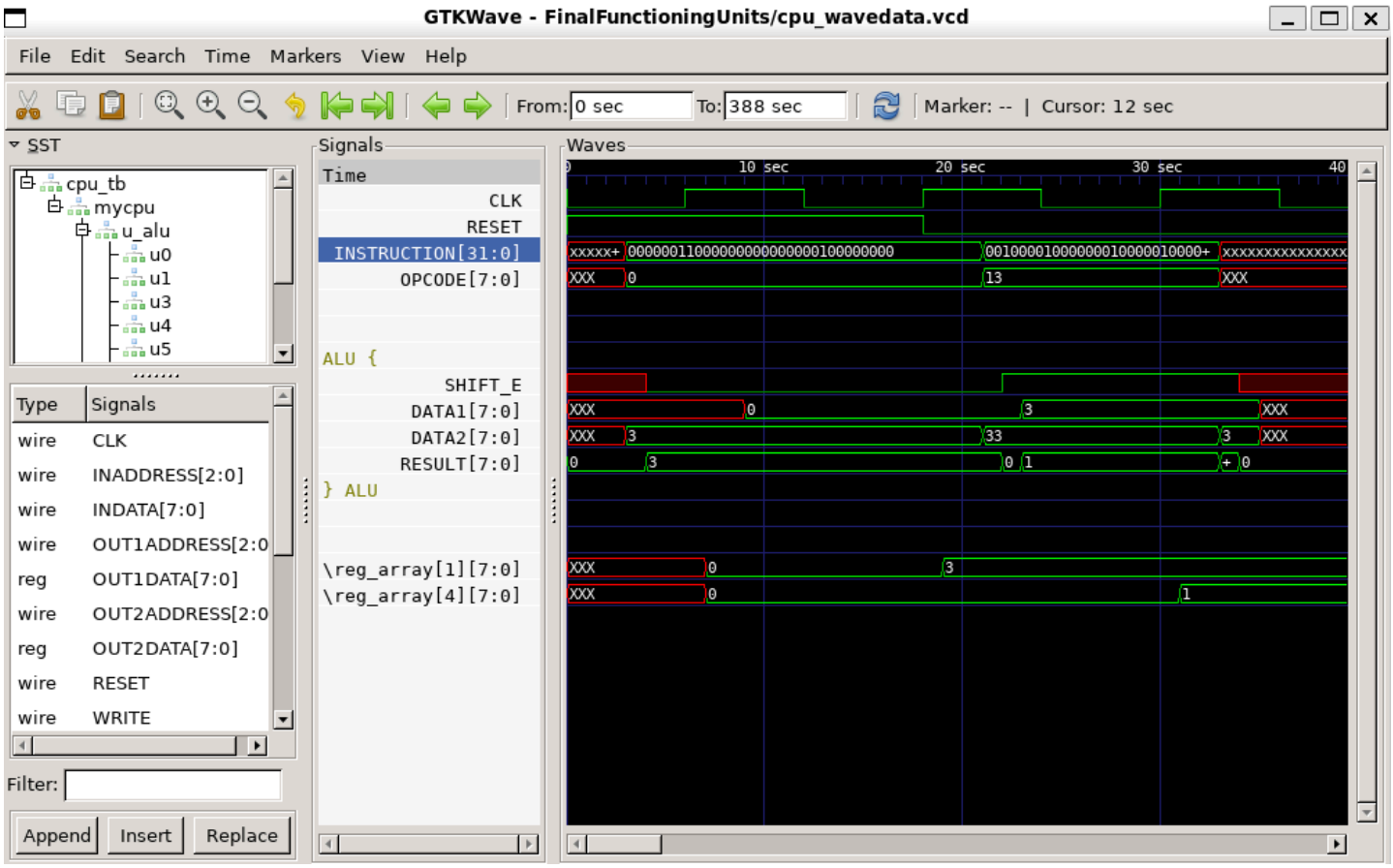

Figure 19: GTK Wave for sra

9.5 ROR

```
loadi  r1, 0x03        // r1 = 3
ror    r1, r1, #1      // r6 = r6 rotated right by 1
```

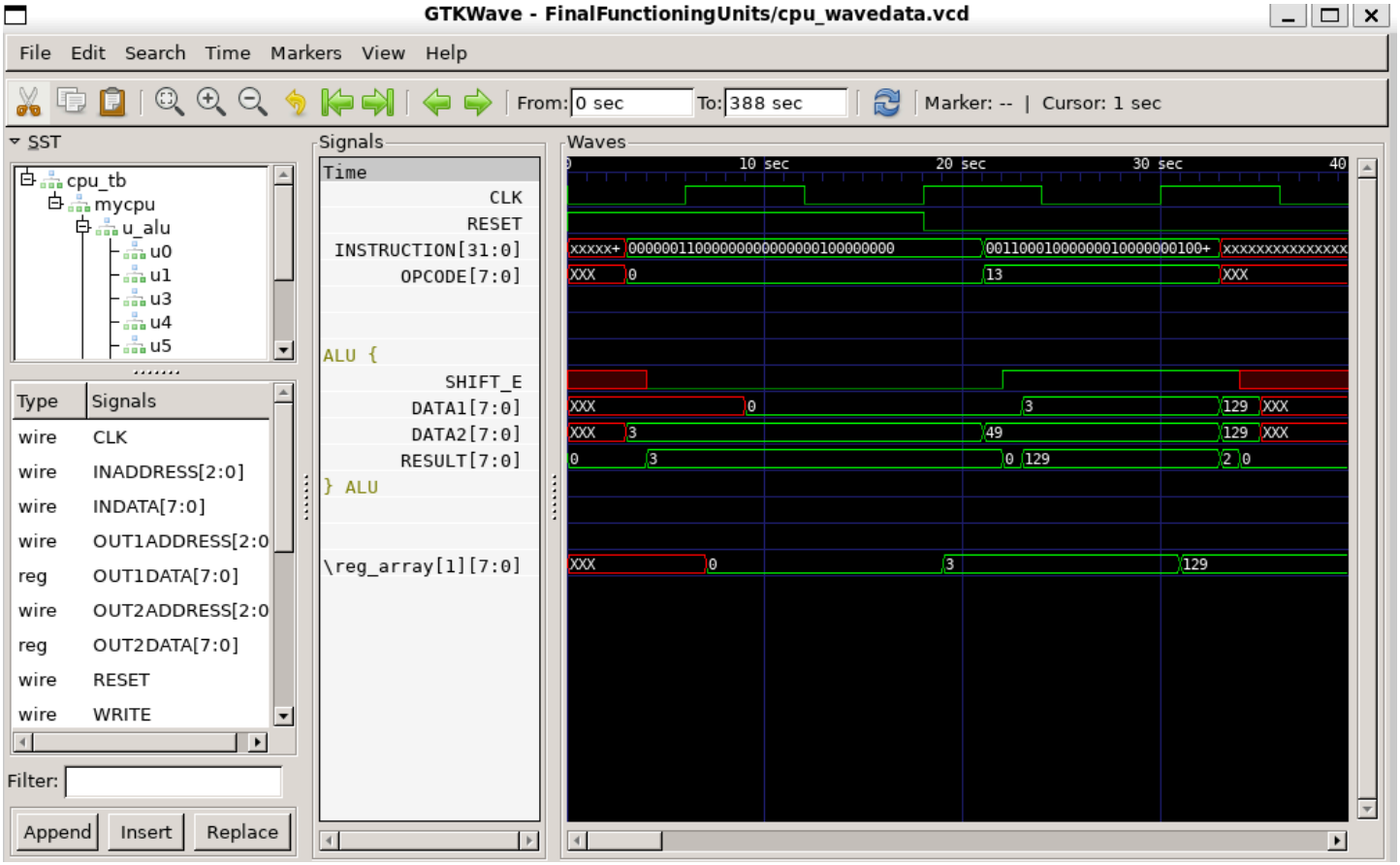

Figure 20: GTK Wave for ror

9.6 BNE

```
loadi  r1, 0x03        // r1 = 3
loadi  r2, 0x01        // r2 = 1
bne    next, r1, r2    // branch if not equal
add r3, r2, r1         // r3 = r2 + r1

next:
sub r4, r1, r2         // r4 = r1 - r2
```

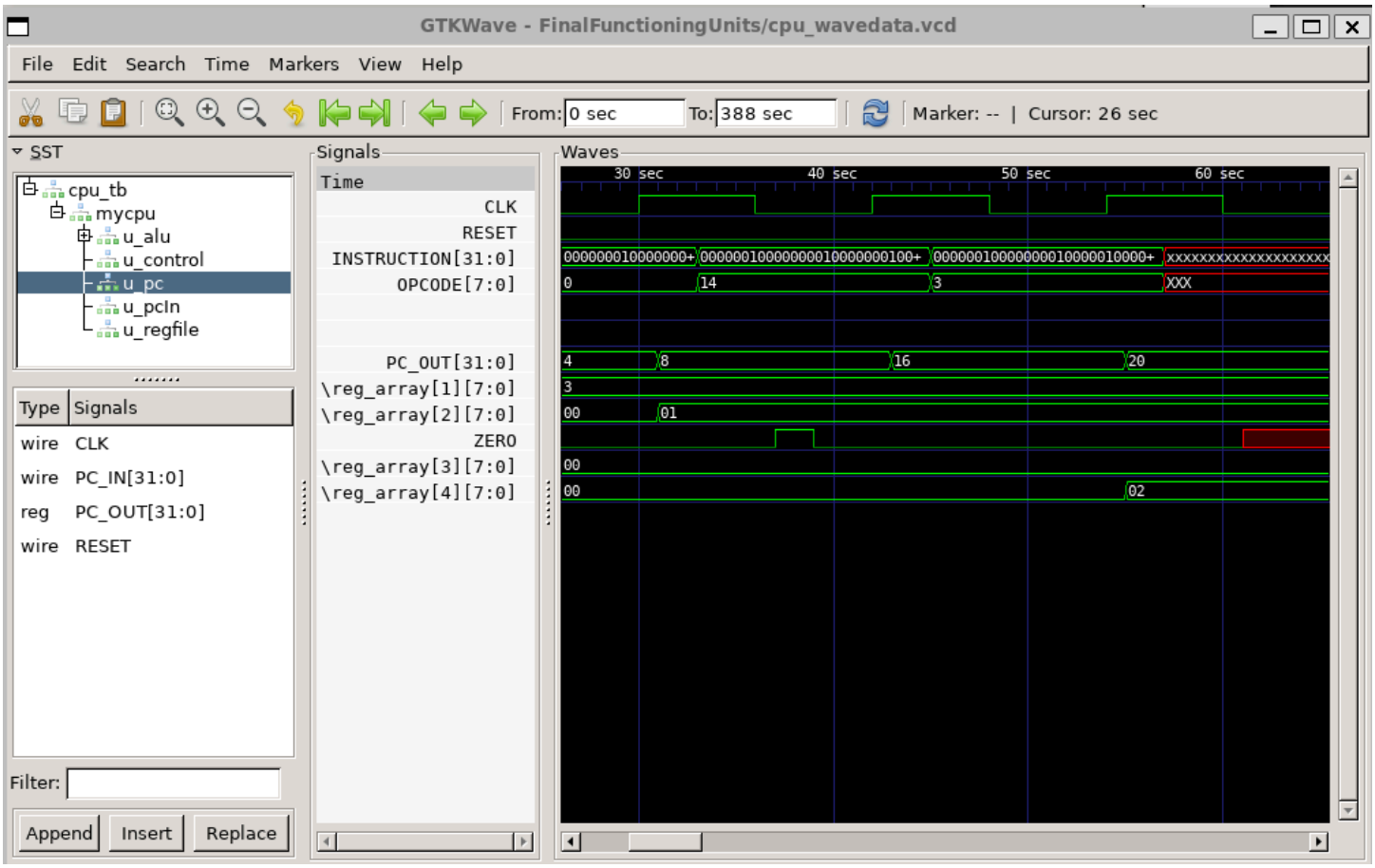

Figure 21: GTK Wave for bne